# Kanea Chatbot

Evaluation Framework & Results

*Metric definitions, methodology, and the latest measured results — for the project report's Evaluation chapter*

**Component Covered:** Full pipeline — intent classifier, query router, danger trigger, RAG retrieval, live analytics

**Reproduce with:** `python backend/eval/evaluate.py`

**Test data:** `backend/eval/test_intents.json`, `test_retrieval.json`

**Version:** 2.0.0

## Table of Contents

## 1. Why One Chatbot Needs Six Metrics

Kanea isn't one model — it's a pipeline: a keyword **intent classifier** feeds a **query router** (static / dynamic / escalation), which decides whether the **RAG retriever** or the **live outage database** supplies context to the **Groq LLM** that generates the reply — with a safety-critical **danger-phrase trigger** able to override all of that. A single accuracy number cannot describe a pipeline like this, so this evaluation scores each stage separately, then adds the operational metrics the app already logs from real usage.

> **Two sources feed this report**
>
> **backend/eval/evaluate.py** — a deterministic, offline script. It needs no Groq API key and makes no network calls, so it always produces the same numbers for the same code and can be re-run as a regression check after any change.
>
> **database.get_analytics()** — live numbers pulled from `chatbot.db`, i.e. from real conversations once the app has been used by real customers.

Run it with:

```
cd backend
python eval/evaluate.py
```

This prints the full report and writes machine-readable results to `backend/eval/results.json`.

## 2. Intent Classification Accuracy

**What it measures:** whether `classify_intent()` (`backend/chatbot.py:93`) assigns the label a human would, across a hand-labeled set of 70 utterances — 10 each for `outage_status`, `fault_report`, `billing`, `safety`, `escalation`, `complaint`, and `general` (`backend/eval/test_intents.json`).

### Formulas

```
Precision(c) = TP(c) / (TP(c) + FP(c))
Recall(c)    = TP(c) / (TP(c) + FN(c))
F1(c)        = 2 · Precision(c) · Recall(c) / (Precision(c) + Recall(c))
Accuracy     = (# correct predictions) / (# total cases)
```

TP/FP/FN are read off a 7×7 confusion matrix (gold label × predicted label), which the script prints in full alongside every mismatched case.

**Why this metric:** intent classification is pure substring keyword matching (`kw in lower`), so it is the layer most exposed to synonyms, slang, and phrasing the keyword lists never anticipated — exactly the gap a labeled test set exposes that reading the code cannot.

| Intent | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| billing | 0.900 | 0.900 | 0.900 | 10 |
| complaint | 1.000 | 0.900 | 0.947 | 10 |
| escalation | 0.889 | 0.800 | 0.842 | 10 |
| fault_report | 0.692 | 0.900 | 0.783 | 10 |
| general | 0.615 | 0.800 | 0.696 | 10 |
| outage_status | 0.778 | 0.700 | 0.737 | 10 |
| safety | 1.000 | 0.700 | 0.824 | 10 |

**Latest result: 81.4% overall accuracy (n = 70)**

Weakest class is **general** (F1 0.696) — *"What can you help me with?"* gets pulled into `escalation` by the word "help", and *"Where is your head office located?"* gets pulled into `outage_status`. **safety** has perfect precision but only 0.700 recall — it loses ground to `fault_report` whenever a hypothetical safety question names a piece of equipment ("transformer", "wiring").

## 3. End-to-End Query Routing Accuracy

**What it measures:** `classify_query_type()` (`backend/chatbot.py:107`), which decides `static` (RAG), `dynamic` (live outage data), or `escalation` (human handoff) — the decision that controls *what data the LLM is even allowed to see*. It is scored using the **predicted** intent from Section 2, not the gold intent, so intent-classifier mistakes propagate exactly as they would in production.

**Formula:** the same accuracy formula as Section 2, over the 3-way route label.

> **Latest result: 87.1% routing accuracy (n = 70)**
>
> Routing accuracy is *higher* than intent accuracy because several intents collapse onto the same route — `billing`, `safety`, `complaint`, `general`, and most of `fault_report` all map to `static` — so an intent mistake does not always cause a routing mistake. Reporting intent accuracy alone would understate how forgiving the routing layer is; reporting routing accuracy alone would hide that the intent label shown on the admin analytics dashboard is less reliable than the routing decision itself.

## 4. Danger / Escalation-Trigger Detection

**What it measures:** `is_danger_message()` (`backend/chatbot.py:129`), which gates the "stay away, call 0800-POWER, `[ESCALATE_TO_AGENT]`" safety path, scored against 5 gold-positive hazard cases mixed into the 70-case set (fallen pole, sparking transformer, smoking meter box, snapped wire, burning smell). A second, compound metric scores the actual production condition, `force_escalate = is_danger_message(text) or query_type == "escalation"`.

### Formulas

```
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1        = 2 · Precision · Recall / (Precision + Recall)
```

#### First run — danger-detection recall was only 40%

Precision 0.333 Recall 0.400 F1 0.364 (TP=2, FP=4, FN=3, TN=61)

The original keyword list missed phrases like "fallen pole", "smoking", and "snapped wire" — and did plain substring matching with no negation/hypothetical handling, so a question *about* hazards ("what hazards should I avoid near overhead lines?") still fired a false positive. **Recall matters more than precision here**: a false negative means a real physical hazard gets an ordinary chatbot reply instead of an immediate warning.

#### Fixed — recall is now 100%

Precision 0.833 Recall 1.000 F1 0.909 (TP=5, FP=1, FN=0, TN=64)

Added the missing hazard phrases, dropped the overly broad bare "burning" keyword (it was false-firing on "the streetlight has been burning nonstop" — just meaning "still on"), and added a guard so FAQ-style hypothetical questions no longer trigger on generic words like "danger"/"hazard". Every real hazard in the test set is now caught, at the cost of one residual false alarm (a hypothetical question containing "electric shock") — a deliberate trade-off: for a safety trigger, a missed real hazard is worse than one extra reminder. See Section 8 for why this residual case was left as-is rather than chased further.

**Compound escalation-trigger metric** (danger OR escalation-intent route):

| | Precision | Recall | F1 | TP | FP | FN | TN |
|---|---|---|---|---|---|---|---|
| Before | 0.667 | 0.667 | 0.667 | 10 | 5 | 5 | 50 |
| After | 0.867 | 0.867 | 0.867 | 13 | 2 | 2 | 53 |

## 5. RAG Retrieval Quality

**What it measures:** `knowledge_base.retrieve_relevant_docs()`, a TF-IDF ranker over the ECG knowledge base, against 16 hand-labeled queries (`backend/eval/test_retrieval.json`) — 15 with a known-correct source section and 1 deliberately off-topic ("jollof rice recipe") to test whether the retriever knows when *nothing* is relevant.

### Formulas (at cutoff k; Kanea uses k = 5 for static queries)

```
Precision@k = |retrieved_k ∩ relevant| / k
Recall@k    = |retrieved_k ∩ relevant| / |relevant|
Hit@k       = 1 if retrieved_k ∩ relevant ≠ ∅ else 0     (averaged over queries)
MRR         = mean( 1 / rank_of_first_relevant_doc )     (0 if none found)
```

| | Recall@5 | Hit@5 | MRR | Precision@5 | Negative-query accuracy |
|---|---|---|---|---|---|
| Before | 0.933 | 0.933 | 0.900 | 0.200 | 0.000 |
| After | 0.933 | 0.933 | 0.900 | **0.463** | **1.000** |

#### First run — strong recall, weak precision, no "I don't know"

The right document was almost always retrieved (Recall@5 93.3%, MRR 0.900). But **Precision@5 was only 0.200**: `top_n` was a fixed count, not a relevance threshold, so 4 of the 5 slots returned per static query were low/no-relevance filler still stuffed into the LLM's system prompt. The off-topic negative case scored 0/1 for the same reason — an irrelevant query ("jollof rice recipe") still returned 5 documents.

### Fixed — added a minimum relevance score

Added `MIN_RELEVANCE_SCORE` so a document must clear a TF-IDF score floor before it counts as "retrieved" at all — not just a rank cutoff. Precision@5 more than doubled to **0.463** and negative-query accuracy went to a perfect **1.000**, while Recall@5/Hit@5/MRR stayed exactly the same (no relevant document got pushed out). The threshold value was chosen by sweeping the test set: 5.0 was the sweet spot; pushing it to 7.0 gains a bit more precision but starts costing recall (down to 0.867), so 5.0 was kept. Also removed the project's own presentation-slide text from the customer-facing corpus (it was being retrieved and handed to the LLM as if it were ECG customer knowledge) — a second contributor to the negative-query fix.

## 6. Live Operational Metrics

Pulled straight from `database.get_analytics()`, already instrumented by the app itself — no new code needed.

| Metric | Source | Formula |
|---|---|---|
| Avg. satisfaction (CSAT) | `chat_ratings.stars` (1–5 via `/chat-rating`) | mean of stars |
| Helpful-feedback rate | `message_feedback.rating` (👍 / 👎 per message) | helpful / (helpful + unhelpful) |
| Escalation rate | `escalations` table vs. total sessions | escalations / total_sessions |
| Intent distribution | `messages.intent` | count per intent, real traffic vs. the test set's even split |

> **Latest snapshot (chatbot.db)**
>
> 16 sessions, 51 user messages, 12 escalations (**75.0%** escalation rate) — but **zero star ratings and zero message feedback**. CSAT and helpful-rate are instrumented but currently unmeasured in practice; report them as "not yet populated," not as a number you don't have. The 75% escalation rate is worth a sentence of explanation too: it's high enough to be worth checking whether it's driven by genuine hazards/human-agent requests or by the danger-keyword false-positive issue in Section 4.

## 7. Qualitative Response Quality (Future Work)

The LLM's generated text itself (tone, faithfulness to retrieved context, conciseness) is not scored by `evaluate.py` — automating that needs either an LLM-as-judge pass or a human rubric, and both require real Groq API calls, which felt out of scope for an offline, free-to-re-run script. The standard approach for a project like this is a small **human-scored rubric**: 5–10 sample conversations, scored 1–5 on *faithfulness* (does it only state facts present in the retrieved context or live outage data), *relevance*, *tone*, and *conciseness* — reported as a mean per criterion rather than a single number.

# 8. Limitations & Recommendations

## 8.1 Addressed — implemented and re-verified against the test sets

| # | Limitation | Fix applied |
|---|---|---|
| 1 | Danger-phrase detection missed real hazards and false-triggered on hypothetical safety questions | Expanded hazard phrases + hypothetical-question guard ( `chatbot.py` ) — Recall 0.400 → 1.000 |
| 2 | RAG retrieval had no relevance floor — always forced 5 documents into the LLM prompt, even for off-topic queries | Added `MIN_RELEVANCE_SCORE` threshold ( `knowledge_base.py` ) — Precision@5 0.200 → 0.463, negative-query accuracy 0 → 1.000 |
| 3 | The project's own presentation slides were retrievable as if they were ECG customer knowledge | Removed presentation-slide text from the customer-facing corpus ( `knowledge_base.py` ) |
| 4 | A page refresh started a brand-new session, so the backend's history-restore feature was never actually used from the same browser | `sessionId` persisted in `localStorage` ; added `GET /session/{id}/messages` so the UI re-hydrates visible history on load |

> **Not implemented as code — a manual step**
>
> Actually using the chatbot and submitting star ratings / thumbs feedback before the report is due, so the CSAT and helpful-rate numbers in Section 6 reflect real usage instead of reading "not yet populated." This can't be faked without fabricating data, which would misrepresent the evaluation.

## 8.2 Deferred — documented as future work, not attempted

Two recommendations were deliberately **not** attempted, because they are architecture changes — new dependencies, no existing training pipeline — with real risk of introducing new bugs on a tight timeline, rather than same-day fixes:

- **Replace the keyword intent classifier with a trained or embedding-based model.** The current classifier is pure substring matching. A small supervised model or embedding-similarity approach would generalize past hardcoded synonyms, but 70 labeled examples is not enough training data to do this reliably, and it would need

validation well beyond a single evaluation run before replacing the classifier the whole pipeline depends on.

- **Move RAG from TF-IDF to semantic embeddings.** Would let paraphrased queries match documents with no literal keyword overlap, but requires a new embedding source (a library like `sentence-transformers`, or an external embeddings API) not currently part of the stack.

### 8.3 Still open — lower priority / longer-term

- Single point of failure on the Groq LLM provider, with no fallback model or cached responses.
- `ADMIN_PASSWORD` falls back to a hardcoded default if the env var isn't set; CORS currently allows all origins; the WebSocket chat endpoint has no authentication.
- No automated test suite / CI gate running `evaluate.py` on every change.
- English-only keyword lists — no Twi/Ga support despite the Ghanaian customer base.

## 9. Summary & Key Findings

| # | Metric | Layer | Before fixes | After fixes |
|---|---|---|---|---|
| 1 | Intent classification accuracy | `classify_intent` | 81.4% (n=70) | 81.4% — unchanged, see 8.2 |
| 2 | Query routing accuracy | `classify_query_type` | 87.1% (n=70) | 87.1% — unchanged, see 8.2 |
| 3 | Danger-detection R / P / F1 | `is_danger_message` | 0.400 / 0.333 / 0.364 | **1.000 / 0.833 / 0.909** |
| 3b | Escalation-trigger P/R/F1 | `force_escalate` | 0.667 / 0.667 / 0.667 | **0.867 / 0.867 / 0.867** |
| 4 | RAG Recall@5 / Precision@5 / MRR / neg. accuracy | `retrieve_relevant_docs` | 0.933 / 0.200 / 0.900 / 0.000 | 0.933 / **0.463** / 0.900 / **1.000** |
| 5 | CSAT / helpful-rate / escalation-rate | live `chatbot.db` | not populated / not populated / 75.0% | not populated / not populated / 75.0% |

> **Three things worth a sentence each in the discussion section**
>
> 1. Danger detection and RAG precision were both fixed and re-verified against the same test sets — a concrete example of evaluation driving a real improvement, not just measuring one.
> 2. Intent classification and routing accuracy are unchanged on purpose: replacing the keyword classifier is a bigger architecture change, deliberately deferred rather than rushed (Section 8.2).
> 3. Live CSAT/feedback metrics are wired up correctly but have no data yet — they need real users rating real conversations before they mean anything.

Re-run `python backend/eval/evaluate.py` any time the code or knowledge base changes — the labeled test sets in `backend/eval/*.json` stay valid as a regression check.